HPE DL360 UEFI BOOTING

Approach Document

Version 1.2

Customer Restricted - Information that is intended for a restricted set of employees with the right / need to know principle relating to a specific Account engagement where information owned by the client but managed by Capgemini or information related to customers and their environments (both Technology and Business related) is being handled. This classification must be used for Capgemini documentation and systems where information is being shared with the client.

Editing		
Verification		
Validation		

Monitoring developments

Version	Date	Editor	Description
V1.0	18-12-2024	Capgemini Engineering Team	Secure boot for HPE DL360 server, Initial Version
V1.1	26-12-2024	Capgemini Engineering Team	Added script file to install all dependencies. Rearranged sections on compilation of kernel modules and adding MOK key for kernel modules.
V1.2	21-01-2025	Capgemini Engineering Team	Added content explaining the need for sbsigntools and efitools, along with steps to verify signatures of GRUB, SHIM, and the kernel under appendix section.

Table 1: Change history

Distribution

Name	Function	Organization
EXFO Engineering Team	Engineering	EXFO
CG Engineering Team	Engineering	Capgemini

Table 2: Distribution list

Content

1 Introduction

1.1 Purpose of the document

The purpose of this document is to provide a comprehensive step-by-step guide to manage Secure Boot on CentOS-7, including generating MOK certificates, signing boot components, and verifying the setup. This ensures system integrity by aligning with UEFI Secure Boot requirements, enabling custom keys and maintaining secure configurations.

1.2 Scope

It covers the process of configuring and enabling Secure Boot on CentOS 7. It details creating custom keys, signing boot files, and verifying Secure Boot status.

1.3 References

- https://lists.centos.org/hyperkitty/list/announce@lists.centos.org/thread/2APH7YUSNGVRJD5LAXBI46BBB5AKWHZZ/
- https://vault.centos.org/7.9.2009/
- How to boot Linux using UEFI with Secure Boot ? — Get Intimate With Cyber ! 0.1 documentation
- https://access.redhat.com/solutions/10154
- https://www.linuxquestions.org/questions/linux-newbie-8/create-iso-from-centos-7-server-4175653114/#google_vignette
- https://www.linuxquestions.org/questions/linux-newbie-8/create-iso-from-centos-7-server-4175653114/#google_vignette

2 Check secure Boot status

After Flashing OS, switch to root user (su in CentOS 7).

1.To check the current Secure Boot status

`mokutil --sb-state`

2.Edit the Base URLs in CentOS-Base.repo

`vi /etc/yum.repos.d/CentOS-Base.repo`

Comment out all mirror URLs by adding a # at the beginning of each line containing mirrorlist.

Add the following baseurl lines under the respective sections:

Under [base]:

`baseurl=http://vault.centos.org/7.9.2009/os/$basearch/`

Under [updates]:

`baseurl=http://vault.centos.org/7.9.2009/updates/$basearch/`

Under [extras]:

`baseurl=http://vault.centos.org/7.9.2009/extras/$basearch/`

Save and exit the file:

Press Esc, then type: wq and hit Enter.

Check the network connection status using

`nmcli dev status`

If disconnected, reconnect using

`nmcli dev connect <interface_name>`

then run the following command to update.

`yum update`

NOTE: Initially, Secure Boot must be under Disabled State.

2.1 Installing Dependencies

Get the files from below link

NOTE: If you don't have wget command, use the command the belowto install wget.

"yum install wget"

wget http://10.16.128.175/New/Dependencies.tar.gz

wget http://10.16.128.175/New/Kernel.tar.gz

wget http://10.16.128.175/New/install-dependencies.sh

wget http://10.16.128.175/New/lastmile-486cf189.tar.gz

wget http://10.16.128.175/New/racer_0x1000_0x2412_0x0001.bit

wget http://10.16.128.175/New/racer_0x1002_0x2412_0x0001.bit

2.2 Dependencies

Provide executable permissions for install-dependencies.sh file.

```
chmod +x install-dependencies.sh
```

Run the script file to install essential tools which involves sbsign , efitools and its dependencies(openssl, dev-tools, libuuid, help2man, perl, git, etc).

```
./install-dependencies.sh
```

2.3 Install sbsign and efi tools:

Install sbsigntools:

- cd Dependencies/secu-os/sbsigntools
- ./autogen.sh
- ./configure
- make check
- make install
- make install check

Install efitools:

- cd ../efitools
- make
- make install

3. Write your own MOK keys

To generate your own MOK certificates, use the following command:

openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der -nodes -days 365 -subj "/CN=Custom MOK Key/”

CN can be any common name

```
[root@localhost exfo]# openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der -nodes -days 3650 -s
ubj "/CN=MOK_KEYS_CG/"
Generating a 2048 bit RSA private key
.....................................................................+++
.....................+++
writing new private key to 'MOK.priv'
-----
[root@localhost exfo]# ls
Dependencies  Dependencies.tar.gz  Kernel.tar.gz  MOK.der  MOK.priv  install-dependencies.sh  lastmile-486cf189.tar.gz
```

MOK.der and MOK.priv will be created.

3.1 Verifying and Resetting MOK Database

Check If MOK keys are already present in MOK database using following command.

mokutil--list-enrolled

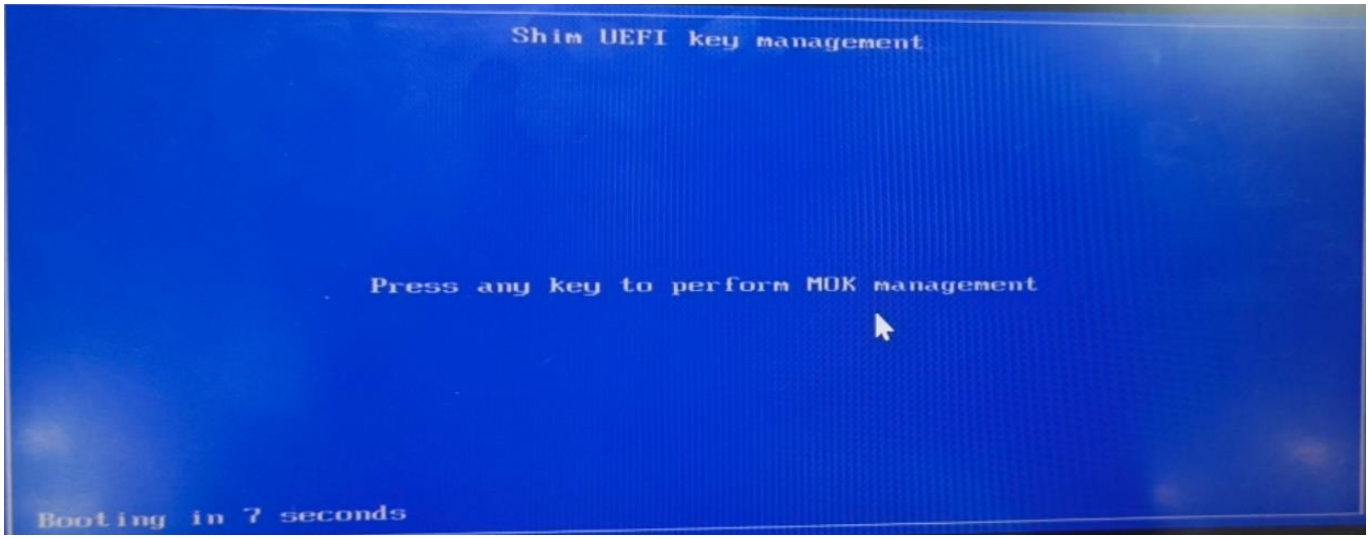
If MOK keys already exists in MOK database. Erase them using following command.

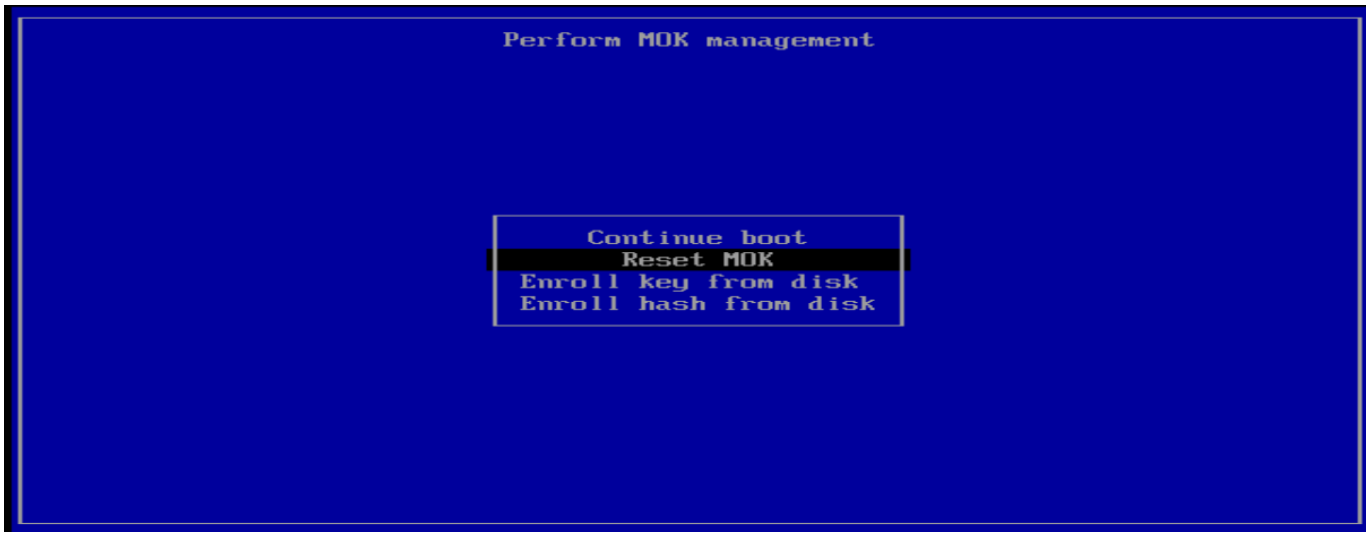
mokutil --reset

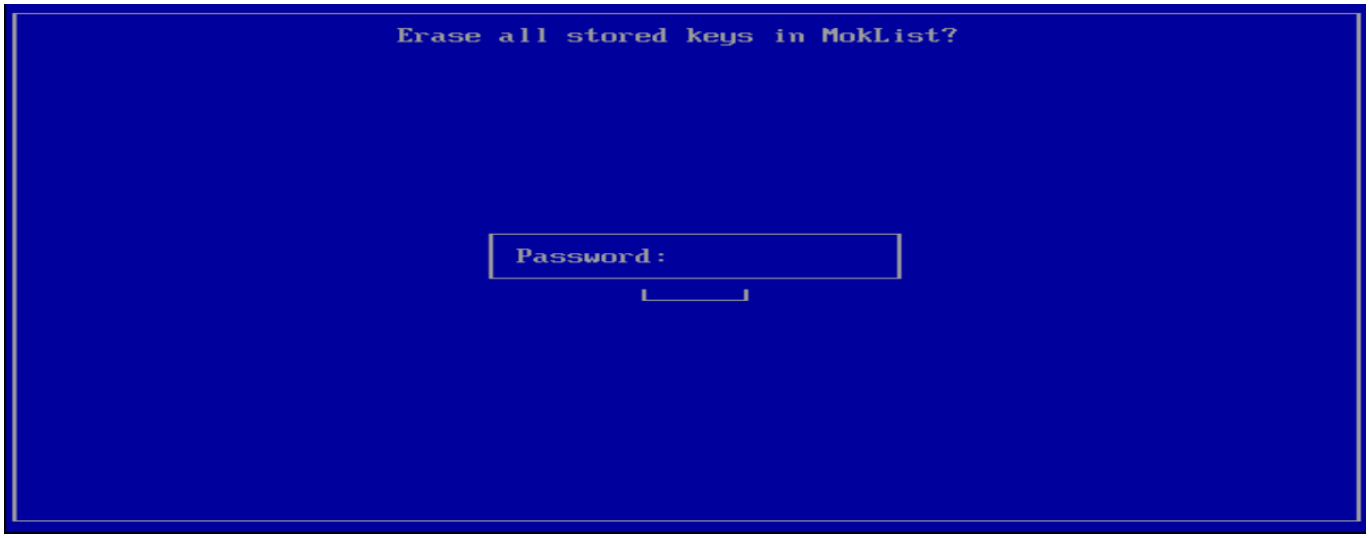
```
[root@localhost exfo]# mokutil --reset
input password:
input password again:
[root@localhost exfo]#
```

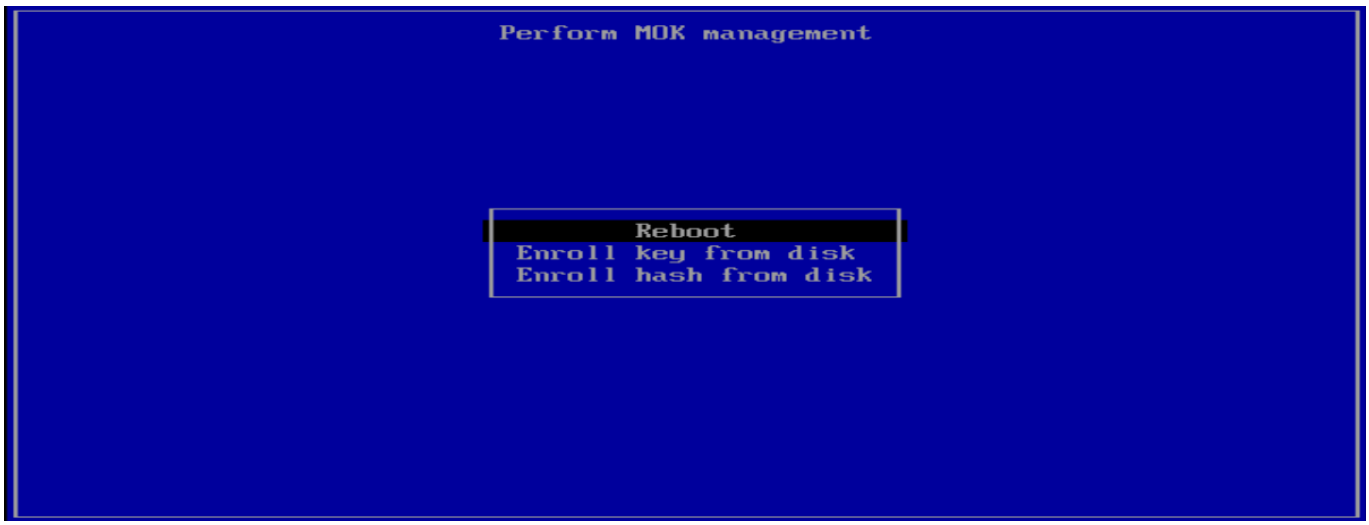
Reboot

Follow MOK management prompts to reset MOK keys.




Perform MOK management
Continue boot
Reset MOK
Enroll key from disk
Enroll hash from disk


Erase all stored keys in MokList?
Password:


Perform MOK management
Reboot
Enroll key from disk
Enroll hash from disk

3.2 Importing MOK keys

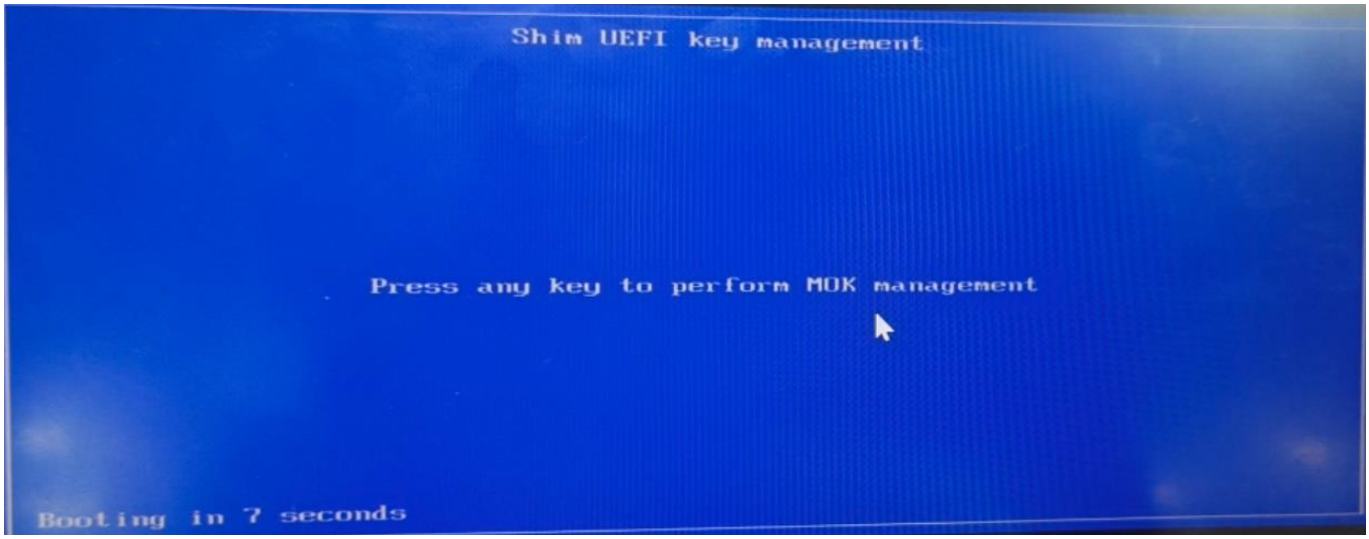
1.Run following command to import the keys:

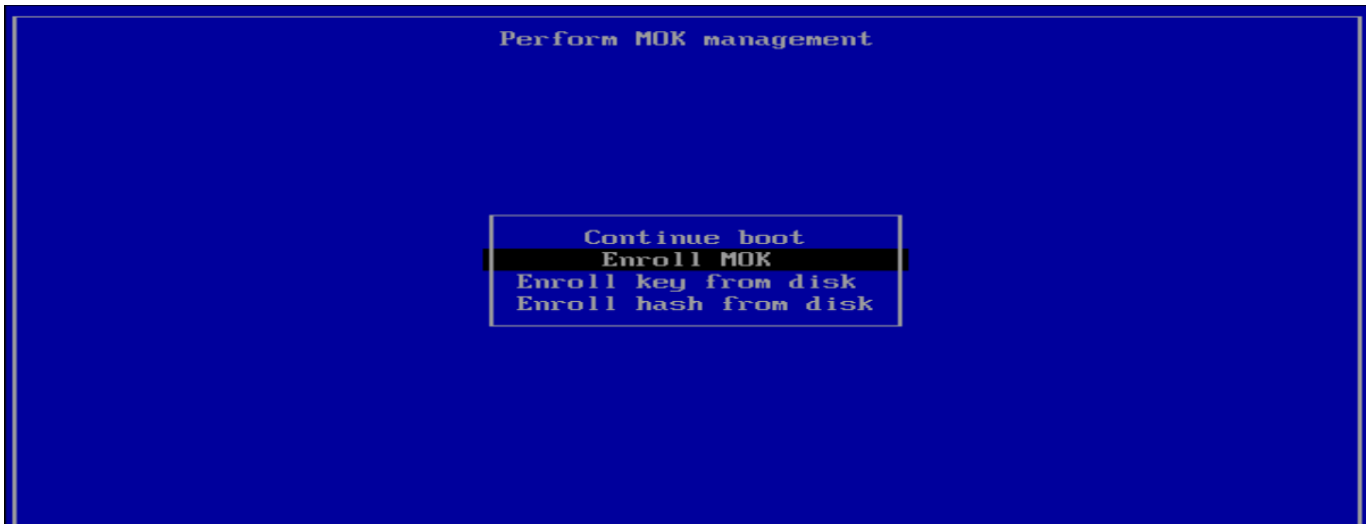
`mokutil - -import /path/to/MOK.der`

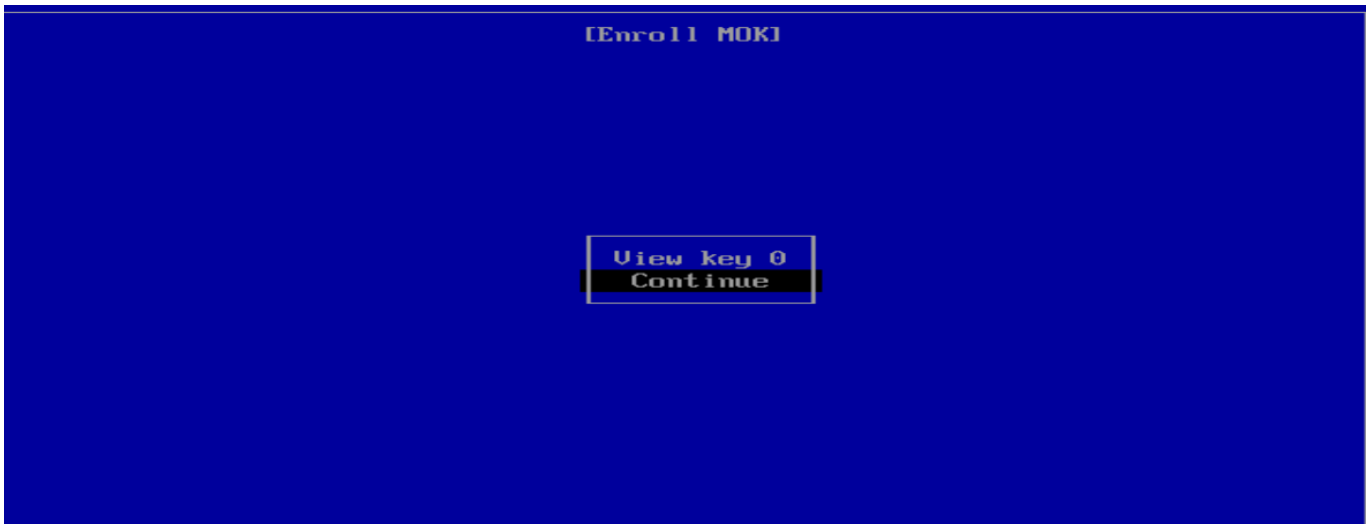
It prompts the password Twice, Provide same password both times.

```
[root@localhost exfo]# mokutil --import MOK.der
input password:
input password again:
[root@localhost exfo]# reboot
```

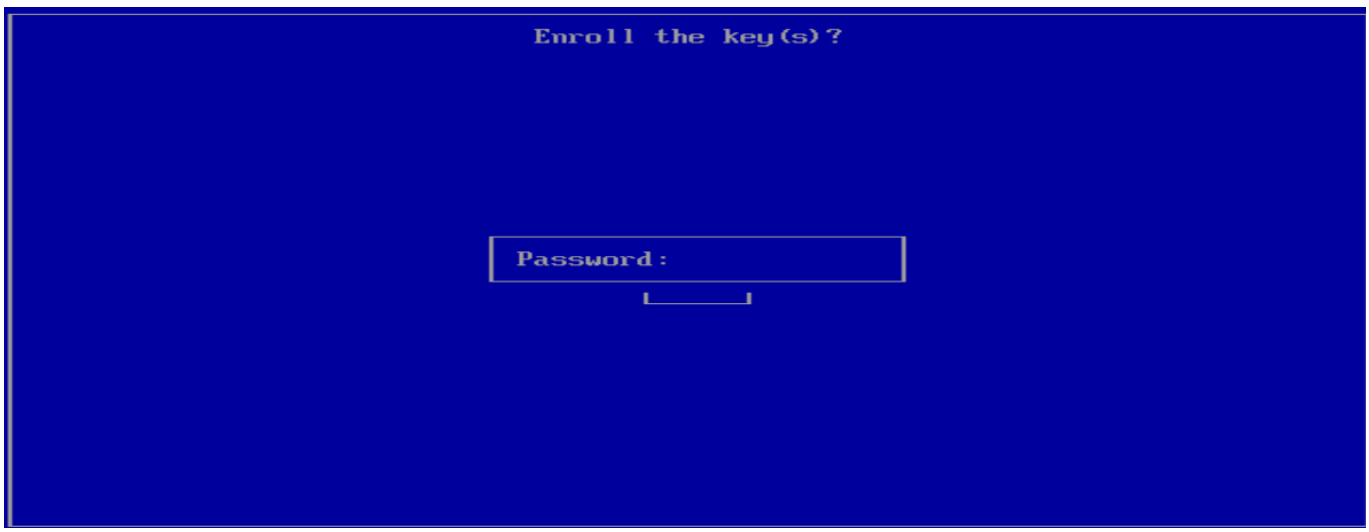
2.Reboot the system and follow on-screen instructions to enroll the MOK keys and provide the password you have created during MOK key importing

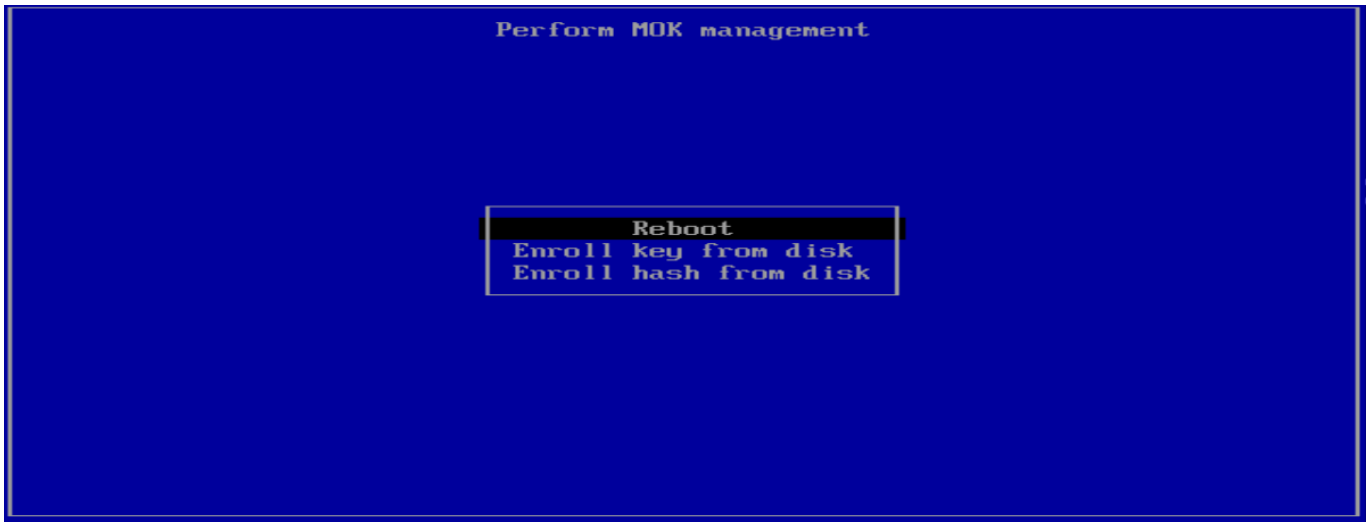





[Enroll MOK]
View key 0
Continue


Enroll the key(s)?
No
Yes


Enroll the key(s)?
Password:


Perform MOK management
Reboot
Enroll key from disk
Enroll hash from disk

4 Building kernel and kernel module

Untar the kernel.tar.gz file and run the following command to install the kernel.

tar -zxvf Kernel.tar.gz

```
[root@localhost exfo]# tar -zxvf Kernel.tar.gz
depKernelSrc/
depKernelSrc/kernel-devel-3.10.0-1160.119.1.el7.x86_64.rpm
```

cd depKernelSrc

rpm -i kernel-devel-3.10.0-1160.119.1.el7.x86_64.rpm

```
[root@localhost exfo]# cd depKernelSrc/
[root@localhost depKernelSrc]# ls
kernel-devel-3.10.0-1160.119.1.el7.x86_64.rpm
[root@localhost depKernelSrc]# rpm -i kernel-devel-3.10.0-1160.119.1.el7.x86_64.rpm
```

Untar the lastmile.tar.gz file.

tar -zxvf lastmile.tar.gz

```
[root@localhost exfo]# tar -zxvf lastmile-486cf189.tar.gz
```

Build the lastmile driver:

cd lastmile/driver

make

```
[root@localhost driver]# make
make -C /lib/modules/3.10.0-1160.119.1.el7.x86_64/build M=/home/exfo/lastmile/driver modules
make[1]: Entering directory `/usr/src/kernels/3.10.0-1160.119.1.el7.x86_64'
  CC [M]  /home/exfo/lastmile/driver/main.o
  LD [M]  /home/exfo/lastmile/driver/lastmile.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/exfo/lastmile/driver/lastmile.mod.o
  LD [M]  /home/exfo/lastmile/driver/lastmile.ko
make[1]: Leaving directory `/usr/src/kernels/3.10.0-1160.119.1.el7.x86_64'
```

sign kernel:

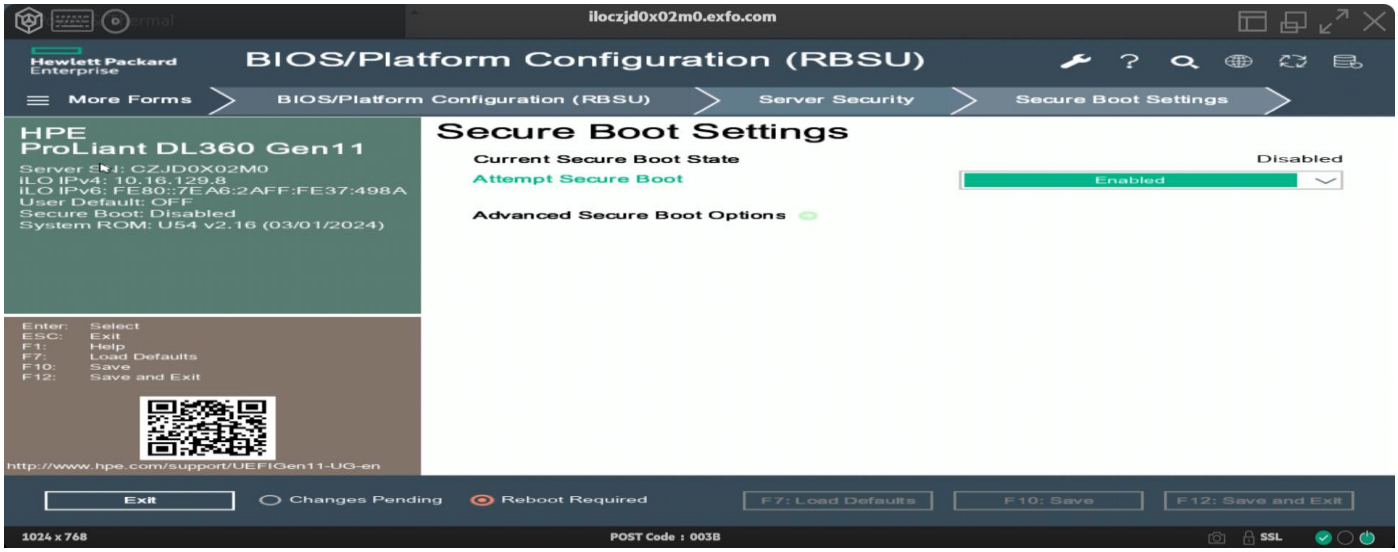
/usr/src/kernels/$(uname -r)/scripts/sign-file sha256 /path/to/MOK.priv /path/to/MOK.der /path/to/lastmile.ko

Verify if the kernel module is signed with your MOK key using the following command:
modinfo lastmile.ko

```
[root@localhost driver]# modinfo lastmile.ko
filename:       /home/exfo/lastmile/driver/lastmile.ko
description:    Driver for Fiberblaze fpga cards
author:         Esa Leskinen <ele@silicom.dk>
license:        not free
retpoline:      Y
rhelversion:    7.9
srcversion:     50D8D4F20D406EF5B319E54
alias:          pci:v00001BC1d00000080sv*sd*bc*sc*i*
alias:          pci:v00001BC1d00000010Esv*sd*bc*sc*i*
alias:          pci:v000010EEd00000007sv*sd*bc*sc*i*
alias:          pci:v00001465d00000006sv*sd*bc*sc*i*
alias:          pci:v00001465d00000005sv*sd*bc*sc*i*
alias:          pci:v00001C2Cd*sv*sd*bc*sc*i*
depends:
vermagic:       3.10.0-1160.119.1.el7.x86_64 SMP mod_unload modversions
signer:         MOK_KEYS_CG
sig_key:        47:89:EB:B4:88:6F:2C:98:42:76:4E:60:CD:51:00:59:BA:54:E5:4E
sig_hashalgo:   sha256
parm:           exclude:charp
parm:           major:uint
[root@localhost driver]#
```

4.1 Enable secure boot

- Reboot the system and press F9 during the boot process.
- Navigate to the following menu:
- **BIOS/Platform Configuration > Server Security > Secure Boot Settings.**
- Attempt to enable Secure Boot by selecting Enabled.



Verify Secure boot status

To confirm Secure Boot is Enabled, Run below command in Centos 7 Terminal.

mokutil --sb-state

```
exfo@project:~$ mokutil --sb-state
SecureBoot enabled
exfo@project:~$ _
```

4.2 Load the kernel module

./load_driver.sh

```
[root@localhost driver]# ./load_driver.sh
[  459.779083] lastmile:  PCIe DMA read/write test is enabled
```

4.3 Generate rawcardtool utility

Build the rawcardtool utility:

cd ../rawcardtool

make clean

make

```
[root@localhost driver]# cd ../rawcardtool/
[root@localhost rawcardtool]# make
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o platform.o platform.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o device.o device.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o register.o register.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o i2c.o i2c.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o gecko.o gecko.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o qspi.o qspi.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o flash.o flash.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o xilinx_bitfile.o xilinx_bitfile.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o cardinfo.o cardinfo.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o cardinfo_savona.o cardinfo_savona.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb   -c -o main.o main.c
gcc -Wall -DVERSION_NUMBER=\""1.3.0"\" -std=c99 -D_POSIX_C_SOURCE=200112L -ggdb -o rawcardtool platform.o device.o register.o i2
c.o gecko.o qspi.o flash.o xilinx_bitfile.o cardinfo.o cardinfo_savona.o main.o -lm
[root@localhost rawcardtool]#
```

If the make command doesn't give the expected output as above, follow these steps:

Clean up old build files:

`make clean`

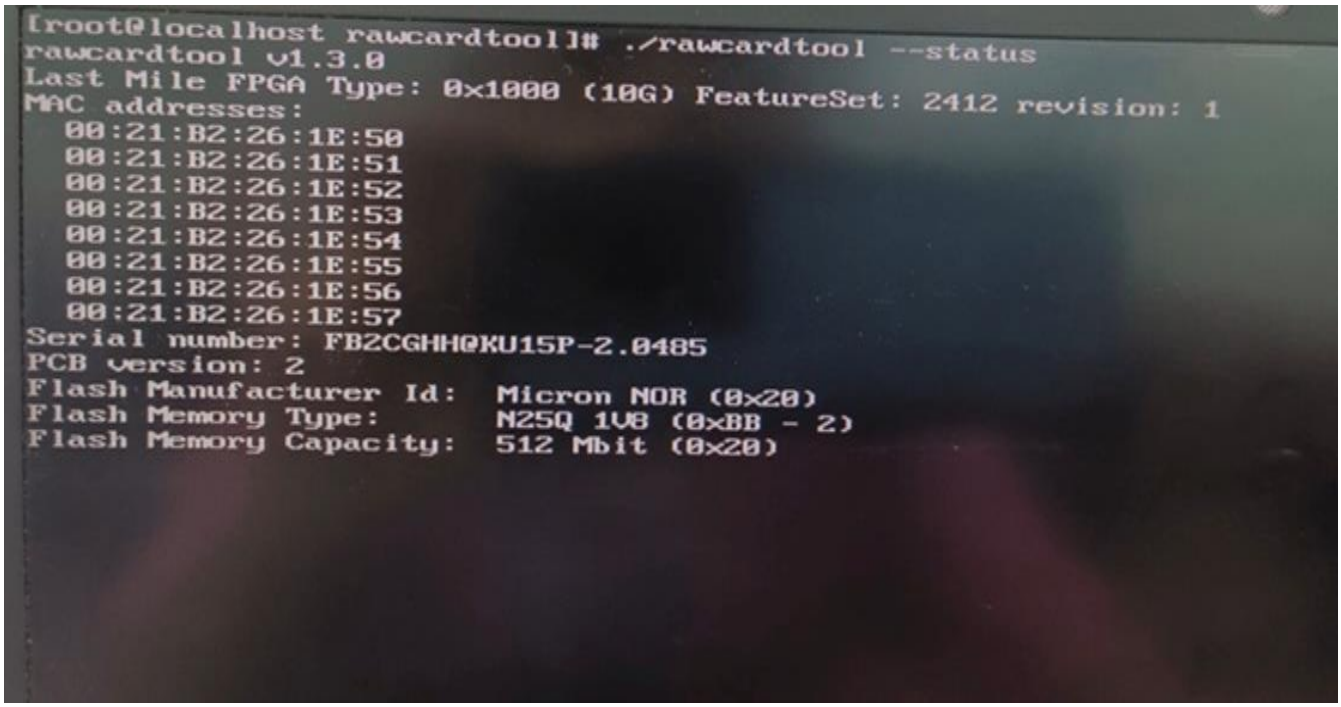
Run the make command again:

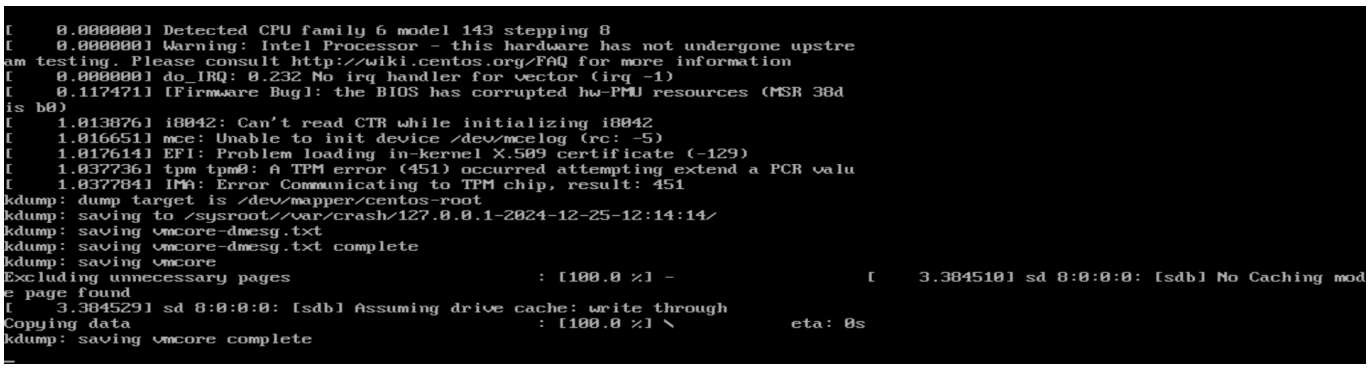
`make`

Verify that everything was successful.

`./rawcardtool --status`

You should see similar output like this:





4.4 Write FPGA bit files

./rawcardtool --flash ../../racer_0x1000_0x2412_0x0001.bit

```
[root@localhost rawcardtool]# ./rawcardtool --flash ../../racer_0x1000_0x2412_0x0001.bit
rawcardtool v1.3.0
Failed to open: /dev/fiberblaze
Failed to open: /dev/fbcard0
Failed to open: /dev/smartnic
Failed to open: /dev/packetmover
Failed to open: /dev/lastmile
Is the driver loaded?
```

If writing bit fails redo the below steps until you see the following output screen.

1. cd lastmile/driver
2. make
3. ./load_driver.sh
4. cd ../rawcardtool
5. make

Write the Bit after redoing above 1-5 steps

./rawcardtool --flash ../../racer_0x1000_0x2412_0x0001.bit

```
[root@localhost driver]# cd ../rawcardtool/
[root@localhost rawcardtool]# maaake
bash: maaake: command not found
[root@localhost rawcardtool]# make
make: Nothing to be done for `all'.
[root@localhost rawcardtool]# ./rawcardtool --flash ../../racer_0x1000_0x2412_0x0001.bit
rawcardtool v1.3.0
Last Mile FPGA Type: 0x1000 (10G) FeatureSet: 2412 revision: 1
Flash Manufacturer Id:  Micron NOR (0x20)
Flash Memory Type:      N25Q 1V8 (0xBB - 2)
Flash Memory Capacity:  512 Mbit (0x20)
Erasing image from flash
Erasing image from flash done
Writing image to flash
Xilinx field data: racer_top;ENCRYPT=YES;COMPRESS=TRUE;UserID=0XFFFFFFFF;Version=2022.1_AR000034237_AR000034053
Xilinx field data: xcku15p-ffve1760-2-e
Xilinx field data: 2024/06/26
Xilinx field data: 17:51:47
Writing image to flash done
Verifying image from flash
Verifying image from flash done
[root@localhost rawcardtool]# _
```

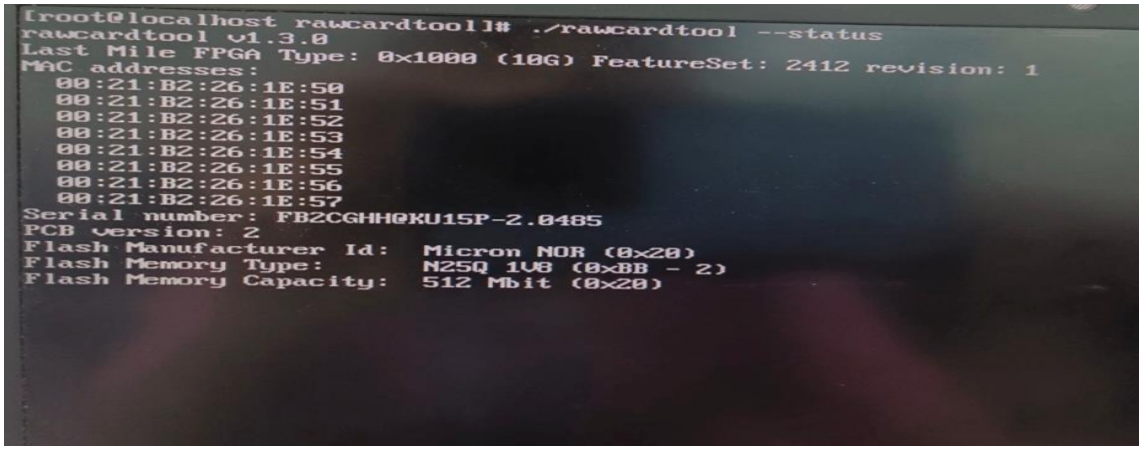
At this point, the server needs to be rebooted in order for the FPGA card to work. Unfortunately, that means redoing steps 1 to 5 under server setup above. This is required because the factory PCI BAR size is 128KB but the new FPGA load has a BAR size of 250MB. Reboot the server by writing into the terminal:

sudo reboot

Verify that the FPGA write was successful:

cd rawcardtool

./rawcardtool –status



Note that the FPGA Type, FeatureSet and revision correspond to the name of the flash file. If you see the same output as before writing the flash, STOP! That means that the card is not properly encrypted, and it loaded the failsafe instead of the primary flash because of that. To continue, the card must be properly encrypted and the

rawcardtool status must show the output above. Once that is done, rawcardtool --status will output the correct text above.

Write the failsafe flash ONLY IF the primary flash was successfully verified in the previous step

./rawcardtool --flashsafe ../../racer_0x1002_0x2412_0x0001.bit

```
[root@localhost rawcardtool]# ls
cardinfo.c          cardinfo_savona.o  flash.c  gecko.o  main.o      qspi.c       README.pdf  xilinx_bitfile.c
cardinfo.h          CHANGELOG.md       flash.h  i2c.c    Makefile    qspi.h       register.c  xilinx_bitfile.h
cardinfo.o          device.c           flash.o  i2c.h    platform.c  qspi.o       register.h  xilinx_bitfile.o
cardinfo_savona.c   device.h           gecko.c  i2c.o    platform.h  rawcardtool  register.o
cardinfo_savona.h   device.o           gecko.h  main.c   platform.o  README.md    VERSION
[root@localhost rawcardtool]# make
make: Nothing to be done for `all'.
[root@localhost rawcardtool]# ./rawcardtool --flashsafe ../../racer_0x1002_0x2412_0x0001.bit
rawcardtool v1.3.0
Last Mile FPGA Type: 0x1000 (10G) FeatureSet: 2412 revision: 1
Flash Manufacturer Id:  Micron NOR (0x20)
Flash Memory Type:      N25Q 1V8 (0xBB - 2)
Flash Memory Capacity:  512 Mbit (0x20)
Erasing image from flash
Erasing image from flash done
Writing image to flash
Xilinx field data: racer_top;ENCRYPT=YES;COMPRESS=TRUE;UserID=0XFFFFFFFF;Version=2022.1_AR000034237_AR000034053
Xilinx field data: xcku15p-ffve1760-2-e
Xilinx field data: 2024/07/24
Xilinx field data: 07:48:08
Writing image to flash done
Verifying image from flash
Verifying image from flash done
[root@localhost rawcardtool]#
```

Reload the FPGA from the flash we just wrote:

cd ..

cd scripts

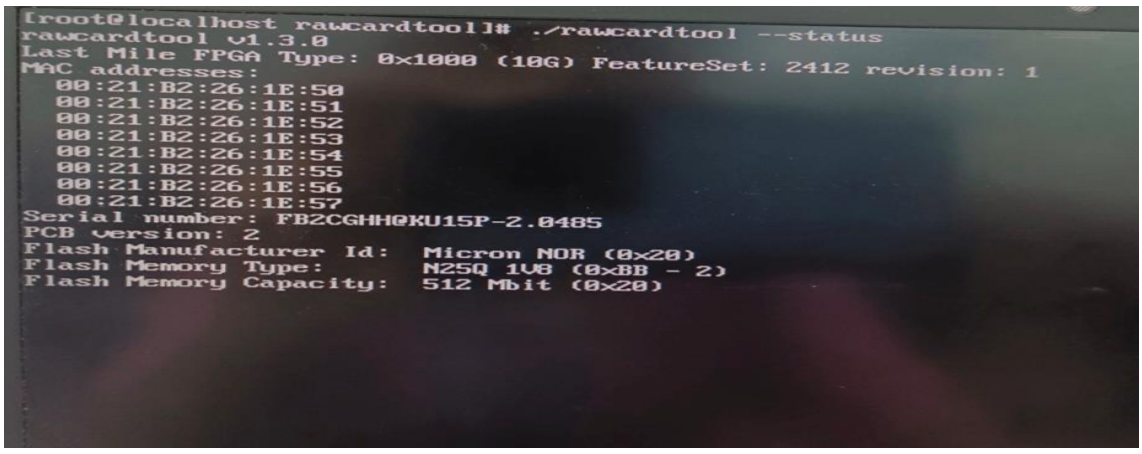
./reload_from_flash.sh failsafe

If failsafe fails redo above 1-5 steps

Verify that it was successful.

cd rawcardtool

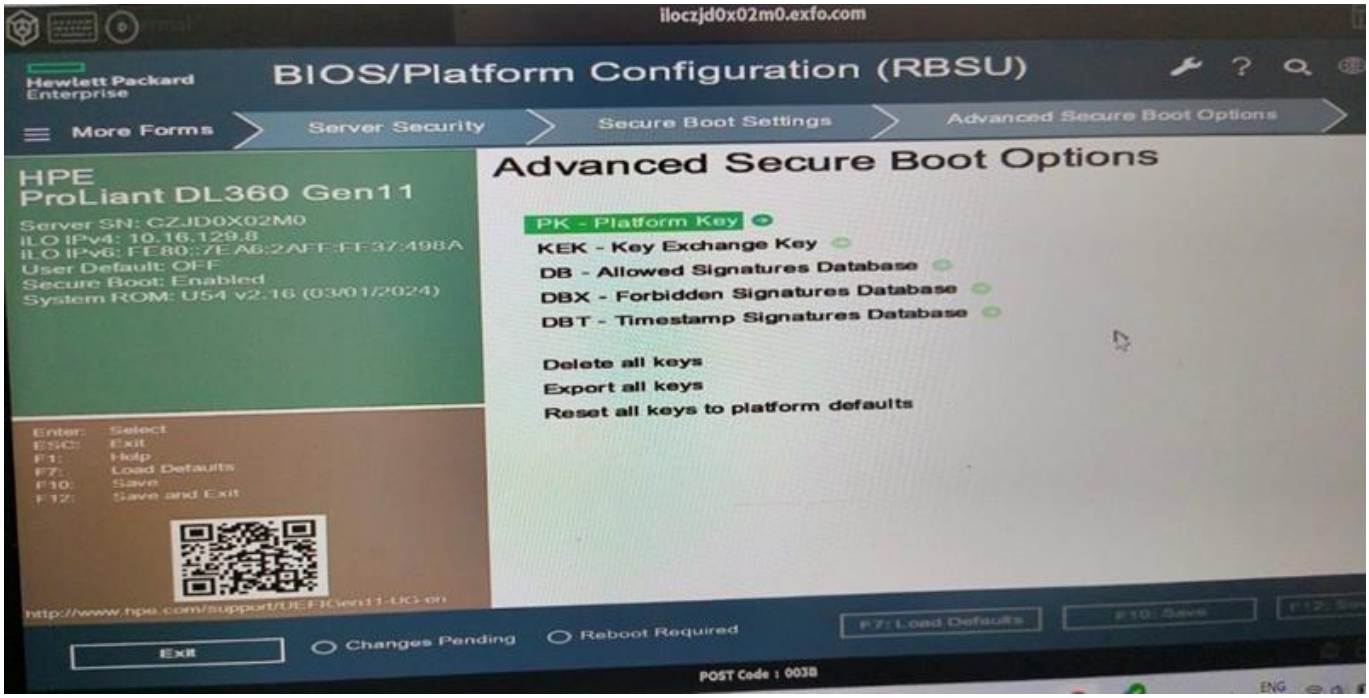
./rawcardtool –status



NOTE: All the above steps need to be performed as a root user.

5 Appendix

To reset all keys to factory default which can clear MOK keys (MOK.der & MOK.priv), use the below option in iLo6 of HPE.



5.1 Signature Verification

In UEFI Secure Boot mode, it is crucial to verify that the Shim, GRUB, and kernel are properly signed and their signatures are valid. This ensures the integrity and authenticity of the boot components.

Verifying the Signatures of Shim, GRUB, and Kernel

To verify the signature of the shim file

Run the following command

`sbverify --list /path/to/shimx64.efi`

you will see the following output indicating signature status

```
[root@localhost exfo]# sbverify --list /boot/efi/EFI/centos/shimx64.efi
warning: data remaining[1111872 vs 1243864]: gaps between PE/COFF sections?
signature 1
image signature issuers:
 - /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft Corporation UEFI CA 2011
image signature certificates:
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft Windows UEFI Driver Publisher
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft Corporation UEFI CA 2011
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft Corporation UEFI CA 2011
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft Corporation Third Party Marketplace Root
```

To verify the signature of grub file

Run the following command

`sbverify --list /path/to/grubx64.efi`

```
[root@localhost exfo]# sbverify --list /boot/efi/EFI/centos/grubx64.efi
signature 1
image signature issuers:
 - /CN=CentOS Secure Boot (CA key 1)/emailAddress=security@centos.org
image signature certificates:
 - subject: /CN=CentOS Secure Boot (key 1)/emailAddress=security@centos.org
   issuer:  /CN=CentOS Secure Boot (CA key 1)/emailAddress=security@centos.org
 - subject: /CN=CentOS Secure Boot (CA key 1)/emailAddress=security@centos.org
   issuer:  /CN=CentOS Secure Boot (CA key 1)/emailAddress=security@centos.org
signature 2
image signature issuers:
 - /CN=CentOS Secure Boot CA 2/emailAddress=security@centos.org
image signature certificates:
 - subject: /CN=CentOS Secure Boot Signing 202/emailAddress=security@centos.org
   issuer:  /CN=CentOS Secure Boot CA 2/emailAddress=security@centos.org
 - subject: /CN=CentOS Secure Boot CA 2/emailAddress=security@centos.org
   issuer:  /CN=CentOS Secure Boot CA 2/emailAddress=security@centos.org
```

To verify the signature of the currently running kernel

Run the following command

`sbverify --list /path/to/vmlinuz-$(uname -r)`

```
[root@localhost exfo]# sbverify --list /boot/vmlinuz-3.10.0-1160.119.1.el7.x86_64
signature 1
image signature issuers:
 - /CN=CentOS Secure Boot (CA key 1)/emailAddress=security@centos.org
image signature certificates:
 - subject: /CN=CentOS Secure Boot (key 1)/emailAddress=security@centos.org
   issuer:  /CN=CentOS Secure Boot (CA key 1)/emailAddress=security@centos.org
 - subject: /CN=CentOS Secure Boot (CA key 1)/emailAddress=security@centos.org
   issuer:  /CN=CentOS Secure Boot (CA key 1)/emailAddress=security@centos.org
signature 2
image signature issuers:
 - /CN=CentOS Secure Boot CA 2/emailAddress=security@centos.org
image signature certificates:
 - subject: /CN=CentOS Secure Boot Signing 201/emailAddress=security@centos.org
   issuer:  /CN=CentOS Secure Boot CA 2/emailAddress=security@centos.org
 - subject: /CN=CentOS Secure Boot CA 2/emailAddress=security@centos.org
   issuer:  /CN=CentOS Secure Boot CA 2/emailAddress=security@centos.org
```

6 Stakeholder

Name	Function	Title
Kesavan Nivetha	Project Manager	Project Manager
Vijayan Bharathi	Delivery Manager	Director
Suresh Kalidasan	Architect	Architect
Navyashree N.V	Software Developer	Engineer
Jhansi Vaddipalli	Software Developer	Engineer

About Capgemini Engineering

Capgemini Engineering combines, under one brand, a unique set of strengths from across the Capgemini Group: the world leading engineering and R&D services of Altran – acquired by Capgemini in 2020 - and Capgemini's digital manufacturing expertise. With broad industry knowledge and cutting-edge technologies in digital and software, Capgemini Engineering supports the convergence of the physical and digital worlds. Combined with the capabilities of the rest of the Group, it helps clients to accelerate their journey towards Intelligent Industry. Capgemini Engineering has more than 52,000 engineer and scientist team members in over 30 countries across sectors including aeronautics, automotive, railways, communications, energy, life sciences, semiconductors, software & internet, space & defence, and consumer products.

Capgemini Engineering is an integral part of the Capgemini Group, a global leader in partnering with companies to transform and manage their business by harnessing the power of technology. The Group is guided every day by its purpose of unleashing human energy through technology for an inclusive and sustainable future. It is a responsible and diverse organization of 270,000 team members in nearly 50 countries. With its strong 50-year heritage and deep industry expertise, Capgemini is trusted by its clients to address the entire breadth of their business needs, from strategy and design to operations, fueled by the fast evolving and innovative world of cloud, data, AI, connectivity, software, digital engineering and platforms. The Group reported in 2020 global revenues of €16 billion.

Get the Future You Want | www.capgemini.com